

SYSTEM DESIGN INTERVIEW

Design Google Image (Reverse Image Search)

given an image, return visually similar / duplicate images from ~100B web images

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Two Complementary Signals
- 06 · ANN Search at 100B Scale
- 07 · Index Sharding & Replication
- 08 · Query Path Deep Dive
- 09 · Storage & Schema
- 10 · ANN Search Algorithms

- [11](#) · [Crawl & Ingestion Pipeline](#)
- [12](#) · [Vector Database Choices](#)
- [13](#) · [Failure Modes & Recovery](#)
- [14](#) · [Design Trade-offs](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

Requirements and scope

Design a **reverse image search** system like **Google Images** or **TinEye**. Given a query image (uploaded bytes or URL), return visually similar and near-duplicate images from a corpus of **~100 billion** web images, ranked by similarity, with sub-200 ms tail latency.

Clarifying Questions

Question	Answer
Corpus size?	~100 billion images crawled from the web
Query type?	Image upload, image URL, or drag-drop in browser
Result type?	Ranked list of visually similar / near-duplicate images
Latency target?	p50 < 80 ms, p99 < 200 ms end-to-end
Query volume?	~1B queries/day (~12K QPS sustained, ~50K peak)
Crawl rate?	~500M new images ingested per day
Recall vs precision?	Top-100 recall > 0.9 vs brute-force ground truth
Languages / OCR?	Out of scope: text-in-image search handled separately

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Search by image	POST /search with image bytes or URL → ranked results
Near-duplicate	Find exact / cropped / rescaled copies (perceptual hash)
Semantic similarity	Find visually-similar (same object class, scene, style)
Result page	Thumbnail, source URL, page title, dimensions
Crawl & ingest	Continuously index new images discovered by web crawler
Re-index	Re-embed corpus when a better model ships (~quarterly)

Non-Functional Requirements

Requirement	Target
Scale	100B images indexed; 1B queries/day
Latency	p99 query latency < 200 ms
Recall	Top-100 recall > 0.9 (vs exact brute force)
Availability	99.95%; degrade with fewer shards rather than fail
Freshness	New crawled image searchable within ~1 hour
Durability	Index sharded + 3-way replicated; rebuildable from raw images

03

Capacity Estimation

Math for scale

Traffic

- Queries/day: **1 billion** → $1e9 / 86,400 \approx \mathbf{11.6K\ QPS}$ sustained
- Peak factor $4\times$ → **~46K QPS peak**
- Crawl ingestion: **500M images/day** → $\sim 5,800$ inserts/sec
- Ratio: query : insert $\approx 2 : 1$ (read-mostly but ingest is non-trivial)

Per-Image Footprint

- Perceptual hash (pHash + dHash + wHash): **~24 bytes** total
- Deep embedding (512-D float32): $512 \times 4 = \mathbf{2,048\ bytes\ (2\ KB)}$
- PQ-compressed embedding (64 sub-vectors $\times$ 1 byte each): **64 bytes**
- Metadata row (URL, title, dims, crawl time): **~512 bytes**
- Thumbnail (128 $\times$ 128 JPEG): **~8 KB**

Storage (100B images)

- Raw embeddings (2 KB): $100B \times 2\ KB = \mathbf{200\ TB}$ (RAM-prohibitive)
- PQ-compressed (64 B): $100B \times 64\ B = \mathbf{6.4\ TB}$ (fits in cluster RAM)
- Perceptual hash table: $100B \times 24\ B = \mathbf{2.4\ TB}$
- Metadata in Bigtable: $100B \times 512\ B = \mathbf{\sim 51\ TB}$
- Thumbnails in object store: $100B \times 8\ KB = \mathbf{\sim 800\ TB}$

Sharding

- Shard at **~100M vectors per shard** → **1,000 shards**
- Per-shard PQ index: $100M \times 64\ B = \mathbf{6.4\ GB}$ (fits in RAM on a 32 GB node)
- $3\times$ replication for HA → **~3,000 index replicas**
- Coordinator fan-out: 1,000 shards behind a single query coordinator

KEY

Key Numbers

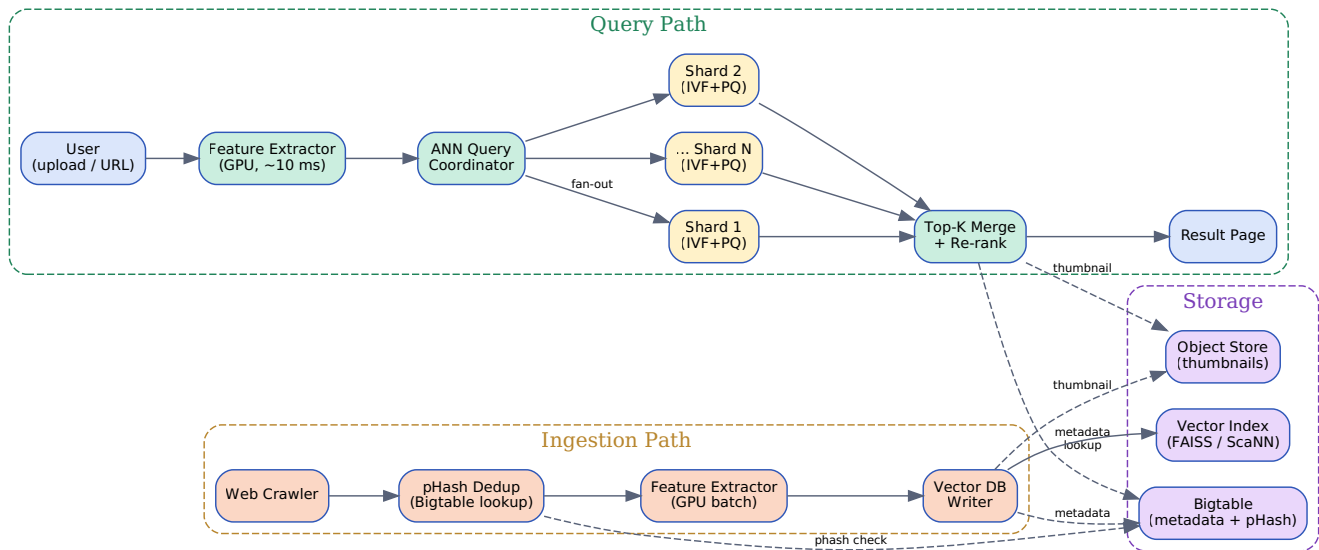
100B images indexed · **1B** queries/day (~46K QPS peak) · **2 KB** raw → **64 B** PQ embedding · **1,000 shards** $\times$ 6.4 GB · p99 < **200 ms**.

04

High-Level Architecture

Query and ingestion paths

Two independent pipelines share one vector index. The **query path** accepts an image, extracts a feature vector on a GPU, fans out to all shards of the ANN index, merges top-K, and re-ranks before returning the result page. The **ingestion path** consumes URLs from the web crawler, deduplicates by perceptual hash, embeds new images, and writes to the vector DB.



Query path (top): image upload → GPU feature extractor → ANN coordinator fan-out → top-K merge → re-rank. Ingestion path (bottom): crawler → pHash dedup → feature extractor → vector DB write.

Component Highlights

- **Feature Extractor:** GPU service running CNN/ViT (e.g., EfficientNet, CLIP) → 512-D float32
- **ANN Coordinator:** stateless; broadcasts query vector to N shards, gathers top-K each
- **Shard:** IVF+PQ FAISS index in RAM; ~100M vectors per shard
- **Re-rank:** exact L2 / cosine on top-1000 with full-precision (or PQ-decoded) vectors
- **Bigtable:** row key = image_id; columns = url, phash, embedding_offset, metadata
- **Object store:** thumbnails for the result page (GCS / S3 with CDN)

05

Two Complementary Signals

Perceptual hashes vs deep embeddings

A reverse image search needs **two distinct similarity notions**: exact / near-duplicate detection (the same JPEG re-saved, cropped, or watermarked) and semantic similarity (different photos of the same object). Perceptual hashes serve the first cheaply; deep embeddings serve the second.

Perceptual Hashes

- **pHash**: 32×32 grayscale → DCT → low-frequency 8×8 → median bit → 64-bit hash
- **dHash**: 9×8 grayscale → adjacent-pixel difference → 64-bit hash
- **wHash**: wavelet-based; robust to compression and resize
- **Distance**: Hamming distance < 5 ⇒ near-duplicate (very high precision)
- **Cost**: ~64 bits per image; fits in Bigtable column; lookups in microseconds

Deep Embeddings

- **Backbone**: CNN (EfficientNet, ResNet) or ViT (CLIP, DINO) penultimate layer
- **Output**: 512-D float32 vector (2 KB raw); L2-normalized for cosine
- **Distance**: cosine similarity > 0.85 ⇒ visually similar
- **Captures**: object class, scene, color palette, composition
- **Cost**: GPU forward pass ~10 ms (batch 32), 2 KB / image at full precision

Comparison

Aspect	Perceptual Hash	Deep Embedding
Captures	Pixel-level near-duplicates	Semantic / visual similarity
Size	~8–24 bytes	2 KB raw / 64 B PQ
Compute	~1 ms CPU	~10 ms GPU
Distance	Hamming (XOR + popcount)	Cosine / L2
Failure mode	Misses recolored / re-shot images	Returns false positives at boundary
Use case	Dedup, copyright, exact-match	Visual search, recommendations

INSIGHT

Why Both

On query, run pHash lookup first (cheap; resolves duplicates with high precision). If the user wanted near-duplicates they get them in < 10 ms. In parallel run the deep-embedding ANN search to surface semantically similar images. Merge both lists with pHash hits ranked first.

06

ANN Search at 100B Scale

IVF, PQ, HNSW

Brute-force scanning 100 billion 2 KB vectors per query would require 200 TB of bandwidth per query — impossible. We need **approximate nearest neighbour (ANN)** structures that prune the candidate set dramatically while preserving recall.

Inverted File (IVF) Index

- **Coarse quantizer:** k-means with K centroids (e.g., $K = 65,536$) trained on a sample
- **Posting list:** each centroid owns the IDs of vectors closest to it
- **Query:** compare query to all K centroids; visit nprobe (e.g., 32) closest lists
- **Speedup:** visits $\sim \text{nprobe}/K$ of the corpus $\rightarrow 32/65,536 \approx 0.05\%$ of vectors

Product Quantization (PQ)

- Split 512-D vector into **M = 64** sub-vectors of 8 dims each
- Train 256 centroids per sub-space $\rightarrow$ each sub-vector = 1 byte ($256 = 2^8$)
- Storage: 64 bytes per vector instead of 2,048 bytes (**32× compression**)
- Distance: precompute query-to-centroid table, look up + sum for each candidate
- Loss: $\sim 3\text{--}5$ percentage points of recall@100 vs full-precision vectors

HNSW (Hierarchical Navigable Small World)

- Multi-layer graph; each node has $\sim M$ neighbours ($M=16\text{--}48$)
- Search: greedy descent from top layer to bottom; logarithmic candidates touched
- **Faster than IVF per shard** ($\sim 5\times$) but uses more RAM (~ 200 B/vector)
- Best for shards ≤ 50 M vectors; combine with PQ as 'HNSW-PQ' if RAM-bound

Comparison: IVF-PQ vs HNSW vs HNSW-PQ

Approach	Memory / vec	Latency	Recall@100
Brute force	2,048 B	minutes	1.00 (ground truth)
IVF-PQ (M=64)	~ 80 B	5–10 ms / shard	0.90–0.93
HNSW (M=32)	~ 200 B + 2,048 B	1–3 ms / shard	0.97

Approach	Memory / vec	Latency	Recall@100
HNSW-PQ	~200 B + 80 B	2–4 ms / shard	0.93

TIP

Production Choice

Use **IVF-PQ** for the 100B-scale main index — only this fits in RAM cluster-wide (6.4 TB total). Reserve **HNSW** for smaller hot indexes (e.g., last-30-days fresh tier or VIP customer datasets) where its lower latency and higher recall justify the RAM cost.

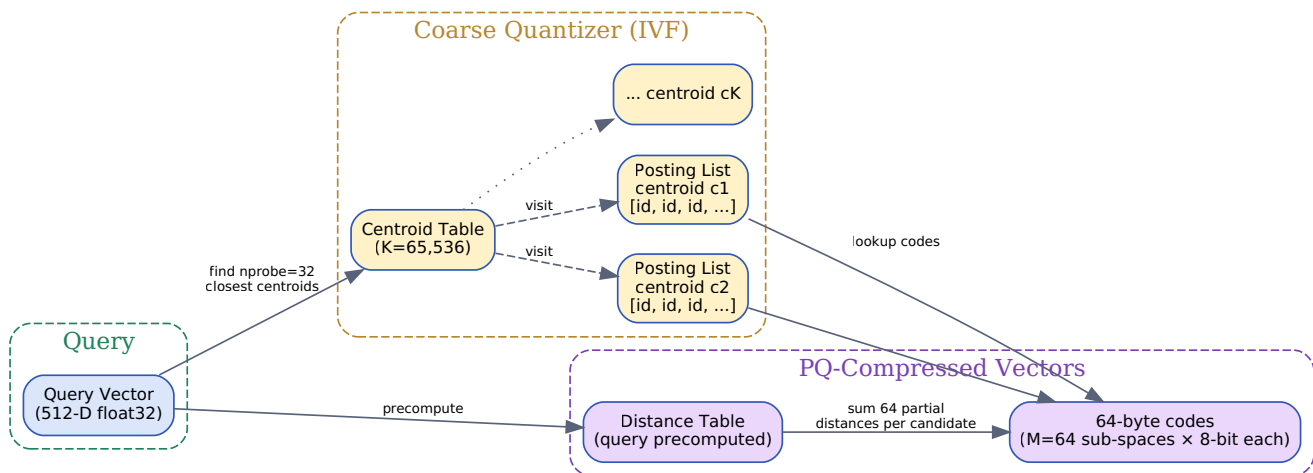
07

Index Sharding & Replication

1,000 shards × 100M vectors

Sharding Scheme

- **Shard key:** $\text{hash}(\text{image_id}) \bmod 1,000$ (random distribution)
- **Why random:** any neighbour can live on any shard → must fan-out to all
- **Why not by feature:** learned-shard routing (SPANN, DiskANN) is complex; random is simpler at this scale
- **Per-shard footprint:** $100\text{M} \times 64 \text{ B} = 6.4 \text{ GB}$ (fits in 32 GB RAM with overhead)
- **Replicas:** 3× per shard for HA + load balancing



Vector index layout per shard: coarse-cluster centroid table → posting list of image IDs → PQ-compressed vectors stored sequentially.

Replication & Placement

- Each shard placed in 3 different racks (and 2 regions for top-tier indexes)
- Coordinator picks healthiest replica per shard at query time (load + latency)
- Hedged requests: if a replica's response > p95, fire a second to another replica
- Failed shard: degrade to N-1 = 999 shards rather than fail the query (see §13)

08

Query Path Deep Dive

From upload to ranked results

End-to-End Steps

1. Client uploads image (or URL): POST /search; max 10 MB; resize server-side to 224×224
2. **pHash compute** (CPU, ~1 ms) → look up in Bigtable; if Hamming < 5, mark as duplicate hit
3. **Feature extraction** (GPU, ~10 ms): batch with other in-flight queries; produce 512-D vector
4. **Coordinator** broadcasts vector to all 1,000 shards (one healthy replica each)
5. **Each shard** runs IVF-PQ search with nprobe=32, returns top-100 (id, approx_distance)
6. **Merge**: coordinator collects $1,000 \times 100 = 100K$ candidates; heap-merge to top-1,000
7. **Re-rank**: fetch full-precision (or PQ-decoded) vectors for the 1,000; compute exact cosine; keep top-100
8. **Hydrate**: Bigtable scan for image_id → metadata; object store for thumbnails (parallel)
9. Return ranked JSON with phash hits first, then semantic neighbours

Latency Budget (p99)

Stage	Budget	Notes
Network in / decode / resize	20 ms	TCP + JPEG decode
pHash compute + Bigtable	5 ms	CPU + cache lookup
GPU feature extract	15 ms	Batched dynamically
Shard fan-out + slowest	60 ms	1,000 RPCs in parallel; tail-tolerant
Top-K merge	10 ms	Heap of 100K candidates
Re-rank exact distances	30 ms	1,000 vectors from cache / shard
Metadata + thumbnail hydrate	40 ms	Parallel Bigtable + object store
Response serialization	20 ms	JSON + network out
Total	~200 ms	Within p99 SLA

INSIGHT

Tail Latency

With 1,000 shard RPCs, the slowest one dominates. Use **request hedging** (fire to a second replica if the first hasn't replied by p95) and a **backup-on-timeout** policy. This keeps the fan-out tail near p95 of a single shard, not p99.9 of 1 in 1,000.

09

Storage & Schema

Bigtable, vector DB, object store

Image Metadata (Bigtable)

```
# Bigtable: image_metadata
# Row key: image_id (16-byte SHA-256 prefix of canonical URL)
#
# Column family 'm' (metadata)
#   m:url          STRING      canonical source URL
#   m:page_url     STRING      page that hosted the image
#   m:title        STRING      page title / alt text
#   m:width        INT64       pixels
#   m:height       INT64       pixels
#   m:mime         STRING      image/jpeg, image/png, ...
#   m:crawled_at   TIMESTAMP    last crawl time
#   m:license      STRING      creative commons, copyright, etc.
#
# Column family 'h' (hashes)
#   h:phash        BYTES(8)     pHash 64-bit
#   h:dhash        BYTES(8)     dHash 64-bit
#   h:whash        BYTES(8)     wHash 64-bit
#
# Column family 'e' (embedding pointer)
#   e:shard_id     INT16        which vector-index shard owns it
#   e:offset       INT64        byte offset of PQ code within shard file
#   e:model_ver    STRING       embedding model version (e.g., v7)
```

Vector Index Storage Layout

```
# Per-shard FAISS index file (memory-mapped)
# Layout (binary):
#
#   header (128 B)      : magic, version, num_vectors, d, M, nlist
#   centroids (K * d * 4) : K=65,536 coarse centroids (float32)
#   pq_codebooks       : M=64 codebooks of 256 centroids each
#   inverted_lists     : posting list per centroid
#       list_id, num_entries, [image_id (8 B), pq_code (64 B)] * N
#
# Per shard: ~100M vectors × (8 B id + 64 B code) = 7.2 GB
# + centroids (16 MB) + codebooks (~512 KB) ≈ 7.2 GB total
# Loaded into RAM at startup; mmap from local SSD on cold start
```

Storage Tiers

Tier	Stores	Tech	Why
Hot index	PQ codes + IVF lists	FAISS in RAM (1,000 shards)	Sub-ms vector ops
Warm full vectors	512-D float32	ScaNN on local SSD	Re-rank top-1,000
Metadata	Image attributes + hashes	Bigtable	Petabyte scale, 10 ms reads
Thumbnails	128×128 JPEGs	GCS / S3 + CDN	Cheap blob, geo-cached
Cold full images	Original bytes	GCS Coldline	Re-embedding on model upgrade

10

ANN Search Algorithms

Pseudocode for IVF-PQ and HNSW

IVF + PQ Search (per shard)

```
def ivf_pq_search(query, index, nprobe=32, top_k=100):
    # 1. Find nprobe closest coarse centroids
    centroid_dists = l2(query, index.centroids)          # K dot products
    probe_lists = argpartition(centroid_dists, nprobe)[:nprobe]

    # 2. Precompute query-to-subspace distance tables
    #   For each of M=64 sub-vectors of the query,
    #   distance to each of 256 sub-centroids.
    dist_table = [] # shape (M, 256)
    for m in range(M):                                  # M=64
        sub_q = query[m*8:(m+1)*8]
        dist_table.append(l2(sub_q, index.codebooks[m]))

    # 3. Scan posting lists, accumulate approximate distance
    heap = MinHeap(top_k)
    for list_id in probe_lists:
        for image_id, pq_code in index.lists[list_id]:
            d = 0.0
            for m in range(M):
                d += dist_table[m][pq_code[m]]          # 1 byte index
            heap.push((d, image_id))

    return heap.top_k()                                # (dist, id) list
```

HNSW Search (per shard, smaller indexes)

```
def hnsw_search(query, graph, ef=64, top_k=100):
    entry = graph.entry_point
    # Greedy descent from top layer to layer 1
    for layer in range(graph.max_level, 0, -1):
        entry = greedy_search(query, entry, layer, ef=1)
    # Beam search at layer 0 with ef candidates
    return beam_search(query, entry, layer=0, ef=ef, top_k=top_k)

def beam_search(query, entry, layer, ef, top_k):
    visited = {entry}
    candidates = MinHeap([(dist(query, entry), entry)])
    results = MaxHeap([(dist(query, entry), entry)])
    while candidates:
```



```

d, node = candidates.pop()
if d > results.top()[0] and len(results) >= ef: break
for nb in graph.neighbours(node, layer):
    if nb in visited: continue
    visited.add(nb)
    dn = dist(query, nb)
    if dn < results.top()[0] or len(results) < ef:
        candidates.push((dn, nb))
        results.push((dn, nb))
        if len(results) > ef: results.pop()
return results.top_k(top_k)

```

Tunable Parameters

- **nprobe (IVF)**: higher = better recall, more compute. 32 is a good starting point
- **M, K (PQ)**: M=64, K=256 → 64 B / vector at >0.9 recall@100
- **ef (HNSW)**: search beam width; 64–128 typical; higher = better recall
- **top_k**: per-shard top-K returned; coordinator merges and re-ranks

11

Crawl & Ingestion Pipeline

From web image to indexed vector

Pipeline Stages

1. **Web crawler** discovers image URLs (HTML , sitemaps); writes to Pub/Sub topic `image_urls`
2. **Fetcher** downloads bytes, validates MIME, rejects > 20 MB or animated GIF frames > 50
3. **pHash dedup**: compute pHash; Bigtable lookup with Hamming-friendly LSH; if duplicate, skip embedding (just add page reference)
4. **Embedding batcher**: queue images for GPU; batch size 64; ~10 ms per batch
5. **PQ encode**: use trained codebooks to compress 2 KB → 64 B
6. **Vector DB writer**: append to shard's posting list (atomic with Bigtable metadata write)
7. **Searchable**: shard reloads its mmap or accepts a write-through delta within ~1 hour

Throughput Math

- Target: **500M images/day** = ~5,800/sec
- After pHash dedup (~30% duplicates): ~4,000 unique embeddings/sec
- GPU throughput: 1,000 imgs/sec/GPU at batch 64 → **~4 GPUs steady state**
- 10× headroom for re-embedding sweeps and bursts → **~40 GPUs reserved**
- PQ training: re-fit codebooks quarterly on a 10M-sample reservoir

Delta vs Full Rebuild

Strategy	When	Cost
Online write-through	Every minute	Cheap; small per-shard append
Hourly compaction	Per shard	Rewrite mmap; <1 GB churn
Quarterly full rebuild	New embedding model	100B × 10 ms = ~1.2 GPU-years
Codebook retraining	Quarterly	10M sample × 1 ms ≈ minutes

WARNING

Model-Version Drift

Vectors from different embedding versions are **not comparable**. Tag every vector with `model_ver`. During a model upgrade, run both indexes in parallel and shadow-test recall before flipping the read traffic. A full 100B re-embedding takes weeks even with 1,000 GPUs.

12

Vector Database Choices

FAISS vs ScaNN vs Milvus

System	Origin	Strengths	Weaknesses
FAISS	Meta	Mature C++ kernels, IVF/PQ/HNSW, GPU support	Library only; you build serving + sharding
ScaNN	Google	State-of-the-art recall/latency Pareto; SIMD-tuned	TF integration heavy; less HNSW support
Milvus	Zilliz	Distributed service out of the box; Kubernetes native	Operational overhead; FAISS under the hood
Vespa	Yahoo	Combined text + vector ranking; multi-tenant	Steeper learning curve; complex DSL
pgvector	Postgres ext.	SQL-native; easy to start	Not suitable beyond ~100M vectors

Recommendation

- Use **FAISS** or **ScaNN** as the per-shard library; build a thin gRPC server around it
- Build the **coordinator** in-house: it's where business logic lives (hedging, multi-tenant routing, re-rank)
- Reserve **Milvus / Vespa** for greenfield projects without a serving infra team
- Avoid **pgvector** at this scale — it can't shard a 100B-vector index

TIP

Build vs Buy

At Google-scale (100B vectors, 1B QPD), no off-the-shelf vector DB is production-ready. Use FAISS / ScaNN as the algorithm, build the serving stack on your own infra (Borg / k8s + Bigtable + Spanner). Mid-scale products ($\leq 1B$ vectors) can succeed with Milvus or Vespa.

13

Failure Modes & Recovery

Graceful degradation

Failure	Impact	Detection	Mitigation
Shard replica down	1/3 of replicas for that shard	Health check; gRPC errors	Coordinator picks healthy replica; auto-rebalance
Whole shard offline	Recall drops ~0.1% (1/1000)	All replicas dead; alert	Degrade to N-1 shards; flag in response
GPU feature extractor down	All queries fail	Inference latency / error rate	Fallback CPU model (lower quality); auto-scale GPU pool
Bigtable hot row	Slow metadata hydrate	p99 hydrate latency spike	Cache hot image_ids in Redis; rebalance tablets
New images unindexed	Recent crawl invisible	Ingestion lag > 1 h	Surface 'fresh tier' separate index; merge top-K
Adversarial near-dup spam	Index pollution; bad results	Cluster-density anomaly	Per-domain rate-limit; pHash bucket caps
Embedding model drift	Recall degrades over time	Offline eval suite	Quarterly re-train + shadow rollout
Coordinator crash	5xx for in-flight queries	Process supervisor	Stateless; restart in seconds; retry idempotent

Graceful Degradation

- **Partial fan-out:** if X% shards slow, return early with what you have; mark response 'partial'
- **Cached top results:** for repeated queries (image_id known) serve from result cache
- **Read-only mode:** stop ingestion if Bigtable degraded; queries continue from prior snapshot
- **Tier fallback:** if deep embedding extractor unhealthy, return pHash-only results with banner

WARNING

Adversarial Inputs

Spammers flood the index with near-duplicates of doorway pages to manipulate ranking. Mitigate with **per-domain quotas**, pHash-cluster size caps (collapse near-dups within a cluster to one canonical), and offline anomaly detection that quarantines suspicious ingestion bursts before they reach the live index.

14

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Index algorithm	IVF + PQ at 100B; HNSW for hot tiers	IVF-PQ saves 32× RAM but loses ~5pt recall vs HNSW. Required for fit.
Sharding	Random by image_id, fan-out to all	Simple and balanced; every query touches all shards. Learned routing is complex.
Two signals	pHash + deep embedding	More compute, but covers both duplicate and semantic search well.
Embedding dim	512-D float32	Higher-D (1024+) costs 2× memory and ~1pt recall gain. Not worth it.
Re-rank pool	Top-1,000 from approximate search	Bigger pool = better quality, more latency. 1,000 hits the knee.
Freshness	1-hour delta + nightly compaction	Lower freshness = bigger compactions, smoother latency. 1 h is humanly fast.
Model upgrade	Quarterly full re-embed	Cost: ~weeks of GPU time. Faster cadence = constant churn; slower = stale.
Result cache	By image_id, 1-day TTL	Saves repeat work for popular queries; some staleness when corpus updates.

15

Interview Playbook

How to present this

Reverse image search is a **vector retrieval** problem dressed up as a search engine. The interview reward comes from articulating the two-signal design and the IVF+PQ math; everything else is standard service plumbing.

45-Minute Outline

1. **Intro (3 min):** 100B images, 1B QPD, p99 200 ms, recall > 0.9
2. **Capacity (5 min):** 2 KB / 64 B vectors, 6.4 TB cluster RAM, 1,000 shards
3. **Two signals (5 min):** pHash (cheap dup) + deep embedding (semantic)
4. **ANN deep dive (10 min):** IVF coarse + PQ compression; nprobe trade-off
5. **Sharding (5 min):** random hash, fan-out, replication, hedged requests
6. **Query path (7 min):** latency budget; tail-latency mitigation
7. **Ingestion (5 min):** crawler → pHash dedup → GPU batch → vector DB
8. **Failures + trade-offs (5 min):** shard down, model drift, adversarial

Key Talking Points

- “ $100B \times 2 \text{ KB} = 200 \text{ TB raw} \rightarrow \text{PQ to } 64 \text{ B} \rightarrow 6.4 \text{ TB fits in cluster RAM}$ ” (size justifies algorithm)
- “Two signals: pHash for duplicates, embeddings for semantics” (covers product surface)
- “Random sharding forces fan-out to 1,000 shards; hedged requests tame the tail”
- “ $n\text{probe} = 32 \text{ of } K = 65,536 \rightarrow \text{visit } 0.05\% \text{ of corpus per shard}$ ”
- “Top-1,000 re-rank with full-precision vectors recovers most PQ recall loss”
- “GPU batch size 64 keeps feature extraction at ~10 ms p99”
- “Quarterly re-embed; tag vectors with `model_ver` to avoid mixing”

Common Follow-ups

- **Q: Why fan-out to all shards?** A: With random sharding, neighbours can be on any shard; learned shard routing (SPANN) reduces fan-out but adds a coarse-shard quantizer that's hard to keep recall on at this scale.
- **Q: How do you handle a new image?** A: Crawler → pHash dedup → GPU batch → PQ encode → append to a delta segment in the matching shard; periodic compaction merges into the main mmap.
- **Q: How do you upgrade the embedding model?** A: Re-embed corpus offline (~weeks on 1,000 GPUs), build a parallel index, shadow-test recall, then flip read traffic. Old vectors stay live until cutover.

- **Q: How do you keep p99 < 200 ms with 1,000 RPCs?** A: Hedged requests against replicas after p95; backup-on-timeout; dynamic batching for GPU; result caching by image_id.
- **Q: What if user uploads adversarial near-dup?** A: pHash-cluster caps and per-domain quotas at ingestion; re-rank scoring penalizes identical pHash clusters dominating the result.
- **Q: Why not HNSW everywhere?** A: HNSW needs the full 2 KB vector + ~200 B graph metadata per vector. At 100B that's 220 TB — won't fit. IVF-PQ trades a few recall points for 30× memory savings.

KEY

Memorize These Numbers

100B images · **1B** queries/day · **2 KB** → **64 B** via PQ (M=64) · **1,000** shards × 6.4 GB · **nprobe=32** of K=65,536 · **p99 < 200 ms** · recall@100 > **0.9**.